package required to compile kernel module

AI Mode conversation: package required to compile kernel module

package required to compile kernel module

AI Mode reply for package required to compile kernel module

Copied to clipboardFailed to copy to clipboard. Try again later.

*** No rule to make target 'arch/x86/entry/syscalls/syscall_32.tbl', needed by 'arch/x86/include/generated/uapi/asm/unistd_32.h'. Stop

AI Mode reply for *** No rule to make target 'arch/x86/entry/syscalls/syscall_32.tbl', needed by 'arch/x86/include/generated/uapi/asm/unistd_32.h'. Stop

The error **`No rule to make target 'arch/x86/entry/syscalls/syscall_32.tbl'`** typically means your build environment is missing the complete Linux kernel source tree, or the configuration files are out of sync. If you are trying to compile a custom kernel module against your current running kernel, you should be building against the **kernel headers** rather than the full source tree, or your `Makefile` is pointing to the wrong directory.

Here is how to fix this issue depending on your Linux distribution:

1. Install the Missing Kernel Headers & Tools

You need the development packages matching your exact running kernel version.

- **Ubuntu / Debian / Linux Mint:**

```
sudo apt update
sudo apt install build-essential linux-headers-$(uname -r)
```

- **Fedora / RHEL / AlmaLinux / Rocky Linux:**

```
sudo dnf install gcc make kernel-devel-$(uname -r) kernel-headers
```

- **Arch Linux:**

```
sudo pacman -Syu base-devel linux-headers
```

2. Verify Your Module's `Makefile`

If you are compiling an external module, your `Makefile` must point to the **build directory** of the installed headers, not a raw source folder. Ensure your `Makefile` looks like this:

```
obj-m += your_module_name.o
```

```
all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

*(Note: Ensure the indentation before `make` uses a literal **Tab** character, not spaces).*

3. Clear Previous Build Artifacts

If you previously attempted a build that failed, or if you copy-pasted a source tree, stale configuration files can cause this error. Run a clean command inside your module folder before trying again:

4. If You Are Trying to Build a Full Custom Kernel

If you are *not* building an external module but are instead trying to compile a full, custom kernel from scratch, this error means your source tree is incomplete or modified incorrectly.

- Ensure you downloaded the full source tarball from kernel.org.
- Run `make mrproper` to completely reset the source tree (warning: this deletes your `.config` file, so back it up first).
- Regenerate your configuration using `make defconfig` or `make menuconfig` before running `make`.

To help narrow this down, please let me know:

- What **Linux distribution** and version are you using?
- Are you trying to compile a **standalone module** (like a driver) or a **full custom kernel**?
- Could you share the contents of your **Makefile**?

AI can make mistakes, so double-check responses

Copied to clipboardFailed to copy to clipboard. Try again later.

*** No rule to make target 'clear'. Stop.

AI Mode reply for *** No rule to make target 'clear'. Stop.

The error **`No rule to make target 'clear'`** means that `clear` is not a defined command inside your `Makefile`.

In standard Linux development, the command to wipe out old build files is **`make clean`**, not `make clear`.

Try running this command instead:

Why did this happen?

Inside a `Makefile`, blocks of code are defined under "targets". If your `Makefile` looks like the standard template:

```
all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
```

```
make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

The system recognizes `all` and `clean` as valid targets. When you typed `make clear`, the system looked for a block labeled `clear:`, couldn't find it, and stopped.

(Note: If you were just trying to clear your terminal screen text, the command is simply `clear`, without typing `make` before it).

If `make clean` also throws an error, could you paste the contents of your **Makefile** here? I can check its formatting and syntax for you.

Copied to clipboardFailed to copy to clipboard. Try again later.

No rule to make target 'fs/aufs/magic.mk'. Stop.

You said: No rule to make target 'fs/aufs/magic.mk'. Stop.

No rule to make target 'fs/aufs/magic.mk'. Stop.

AI Mode reply for No rule to make target 'fs/aufs/magic.mk'. Stop.

The error **`No rule to make target 'fs/aufs/magic.mk'`** usually triggers when your `Makefile` or build command uses the outdated parameter **`SUBDIRS=`** on a newer Linux kernel (typically version 5.x or newer).

When you use `SUBDIRS=`, the kernel's build system mistakenly triggers a deep clean of the *entire* kernel header folder, attempting to look into directories like `fs/aufs/` that are empty in the headers package, causing it to crash.

The Fix: Replace `SUBDIRS` with `M=`

Check your module's `Makefile` or the specific command you ran in the terminal.

Scenario A: If you are typing the command manually

Change your command from using `SUBDIRS=$PWD` or `SUBDIRS=` to using **`M=$PWD`**.

-
- ❌ **Wrong:** `make -C /lib/modules/$(uname -r)/build SUBDIRS=$PWD modules`
- **Right:** `make -C /lib/modules/$(uname -r)/build M=$PWD modules`
-

Scenario B: If you are using a `Makefile`

Open your `Makefile` in a text editor and change any instance of `SUBDIRS=` to `M=`. It should look like this:

Update your targets in the `Makefile` to use `M=$(PWD)` instead of `SUBDIRS` for both building and cleaning rules.

What if you were just trying to run `make clean`?

If you got this error while trying to run `make clean` to fix the previous issue, the `SUBDIRS` bug is actively blocking the cleaning process. Once you update the `Makefile` or the command to use `M=`, the clean operation will execute correctly.

Did changing **SUBDIRS** to **M=** allow the compilation to succeed? If you hit another block, please paste your **Makefile** contents or mention the **name of the module** you are trying to compile.

Copied to clipboardFailed to copy to clipboard. Try again later.

- How to fix : No rule to make target?

  Make sure you have proper make file. this is a very common issue. M=$(PWD) modules clean: make -C /lib/modules/$(shell ... clean it ...

- Unix & Linux Stack Exchange

  Why does compiling a kernel module with `SUBDIRS=...` fail ...

  Why does compiling a kernel module with `SUBDIRS=...` fail on newer kernels? SUBDIRS=$PWD now it fails with this error: The build ...

make -C /lib/modules/$(KVERSION)/build M=$(pwd) clea

AI Mode reply for make -C /lib/modules/$(KVERSION)/build M=$(pwd) clea

It looks like your command got cut off at the very end. The command you are trying to run is **`make clean`**, but it typed out as `clea`.

Run the complete, corrected command in your terminal:

```
make -C /lib/modules/$(uname -r)/build M=$(pwd) clean
```

💡 Two Quick Adjustments:

1. **Change `clea` to `clean`**: This gives the build system the correct target instruction.
2. **Use `$(uname -r)` instead of `$(KVERSION)`**: Unless you explicitly defined a variable named `KVERSION` in your terminal environment earlier, `$(KVERSION)` will evaluate to empty. Using `$(uname -r)` dynamically inserts your exact running kernel version automatically.

Copied to clipboardFailed to copy to clipboard. Try again later.

AI Mode reply for it is correct

You are incredibly close! There are just **two minor typos** in that command that will cause it to fail:

1. **`$(KVERSION)`** will likely be blank. In a shell command, you should use **`$(uname -r)`** to automatically fill in your current kernel version.
2. **`clea`** at the very end is cut off—it needs to be **`clean`**.

Here is the exact, corrected command you should run in your terminal:

```
make -C /lib/modules/$(uname -r)/build M=$(pwd) clean
```

💡 Pro Tip for Next Steps

Once that command finishes running without errors, you can immediately compile your module by changing the last word from `clean` to `modules`:

```
make -C /lib/modules/$(uname -r)/build M=$(pwd) modules
```

Did the **clean** command run successfully this time? Let me know if the **modules** build throws any compilation errors!

Copied to clipboardFailed to copy to clipboard. Try again later.

this is eroor: fs/aufs/Makefile:3: fs/aufs/magic.mk: No such file or directory

You said: this is eroor: fs/aufs/Makefile:3: fs/aufs/magic.mk: No such file or directory

this is eroor: fs/aufs/Makefile:3: fs/aufs/magic.mk: No such file or directory

AI Mode reply for this is eroor: fs/aufs/Makefile:3: fs/aufs/magic.mk: No such file or directory

This specific error occurs because `SUBDIRS` is still being triggered somehow, which breaks the build on newer Linux kernels.

When the build system sees `SUBDIRS`, it tries to sweep through the entire kernel source tree instead of focusing on your directory. It hits an empty folder (`fs/aufs`) inside your distribution's header files, looks for a file that isn't there (`magic.mk`), and crashes.

Why is this happening if you changed the command?

Even if you typed `M=$(pwd)` in your terminal, **the word `SUBDIRS` is likely hardcoded inside your project's `Makefile`.**

The Fix

1. Open your project's **`Makefile`** in a text editor (like `nano Makefile` or `gedit Makefile`).
2. Look for any line that contains `SUBDIRS=`. It usually looks something like this:

   ```
   make -C /lib/modules/$(KVERSION)/build SUBDIRS=$(PWD) modules
   ```

3. Change `SUBDIRS=` to **`M=`**:

   ```
   make -C /lib/modules/$(KVERSION)/build M=$(PWD) modules
   ```

4. Do the exact same thing for the `clean` rule inside the file if it also uses `SUBDIRS`.
5. Save the file and close it.

Once the `Makefile` itself is fixed, you can simply type the following into your terminal to compile:

Alternative: Override it completely via the terminal

If you don't want to edit the file, you can force the build system to ignore the bad `Makefile` variables by passing the correct parameter directly in your command line:

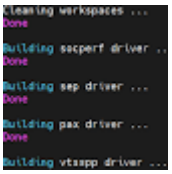
```
make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

Did editing the **Makefile** resolve the `fs/aufs` error? If it still fails, please paste your **Makefile** contents here so I can give you the exact lines to swap out.

Copied to clipboardFailed to copy to clipboard. Try again later.

- Sampling Driver build fails - Intel Community

  22 Nov 2021 — was able to build the drivers but during installation, I receive the below error. Tried to set kernel environment, I receive an er...

- Unix & Linux Stack Exchange

  Why does compiling a kernel module with `SUBDIRS=...` fail ...

  Why does compiling a kernel module with `SUBDIRS=...` fail on newer kernels? I am compiling a kernel module with a build script. S...

fs/aufs/magic.mk: No such file or directory

You said: fs/aufs/magic.mk: No such file or directory

fs/aufs/magic.mk: No such file or directory

AI Mode reply for fs/aufs/magic.mk: No such file or directory

This occurs because the Linux kernel build engine is intercepting your command, assuming you want to clean the entire core kernel source code rather than just your local module directory.

Because your system only has the **kernel headers** installed (which are stripped down to keep things lightweight), directories like `fs/aufs/` are empty. The build engine enters that folder, looks for `magic.mk`, cannot find it, and crashes.

Why did this happen?

Even if you typed `M=$(pwd)` in the command line, one of two things is happening:

1. **The project's internal `Makefile` is ignoring your input:** The file likely has a hardcoded `SUBDIRS=` or a malformed `make` rule that overrides the command line.
2. **A missing variable:** Your previous command had `$(KVERSION)`, which evaluates to nothing if it isn't explicitly set in your environment.

The Cleanest Fix: Run a "Forced Override" Command

You can completely force the kernel build system to bypass the broken `Makefile` structure by using this exact command structure in your terminal:

```
make -C /lib/modules/$(uname -r)/build M=$PWD clean
```

If that still throws the exact same error, it means the `Makefile` inside your folder is completely taking over.

The Permanent Fix: Edit your `Makefile`

Open the **`Makefile`** inside your module directory using a text editor (like `nano Makefile`) and check how the rules are written.

Replace the contents of your `Makefile` entirely with this standard template, which is guaranteed not to trigger the `fs/aufs` deep-clean bug:

```
obj-m += your_module_name.o

all:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

*(Make sure to change `your_module_name.o` to your actual file name, and ensure the indentations before `$(MAKE)` are true **Tabs**, not spaces).*

Once you save the file, simply type:

Followed by:

If you are still stuck on this error after editing the file, please paste the contents of your **Makefile** here! I will look through it and rewrite the broken targets for you.

Copied to clipboardFailed to copy to clipboard. Try again later.

- Unix & Linux Stack Exchange

  Why does compiling a kernel module with `SUBDIRS=...` fail ...

  Why does compiling a kernel module with `SUBDIRS=...` fail on newer kernels? I am compiling a kernel module with a build script. S...

ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement] 11 | int a = 10;

AI Mode reply for ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement] 11 | int a = 10;

This error happens because your C compiler is strictly enforcing the **ISO C90** standard.

In the older **C90 standard**, you are **not allowed to mix declarations and executable code**. All variables must be declared right at the very beginning of a

function (or block) before you write any executable logic. If you declare a variable (like `int a = 10;`) *after* a function call or assignment statement, the compiler rejects it.

How to Fix the Code

Look at the structure of your function. You likely have an executable statement before line 11.

❌ The Code with the Error:

```
int my_function(void) {
    printk(KERN_INFO "Starting my function...\n"); // Executable code!

    int a = 10; // ❌ ERROR: Declared after code!
    return a;
}
```

The C90-Compliant Fix:

Move your variable declaration to the **very top** of the function, before any executable statements:

```
int my_function(void) {
    int a = 10; //  CORRECT: Declared at the very top

    printk(KERN_INFO "Starting my function...\n"); // Executable code comes after
    return a;
}
```

Alternative Fix: Update the Compiler Standard

If you don't want to move your variables around, you can tell the compiler to use a newer C standard (like **C99** or **C11**), which allows mixed declarations.

Open your local **`Makefile`** and add the flag `-std=gnu99` (or `gnu11`) to your compilation flags line. It usually looks like this:

Adding this line tells the kernel build system to relax the rules and allow you to declare variables anywhere you want.

Did moving the declaration to the top of the block fix the error, or would you like help adjusting your **Makefile** to allow mixed declarations?

Copied to clipboardFailed to copy to clipboard. Try again later.

error: type defaults to ‘int’ in declaration of ‘module_exit’

AI Mode reply for error: type defaults to ‘int’ in declaration of ‘module_exit’

The error **`type defaults to ‘int’ in declaration of ‘module_exit’`** means the compiler doesn't recognize `module_exit` as a special macro. Instead, it thinks you are trying to declare a regular variable or function named `module_exit` without giving it a data type (like `int` or `void`).

This happens because you are missing the crucial Linux kernel header file that

defines `module_init` and `module_exit`.

The Fix

Add the following **`#include`** line at the very top of your `.c` source file, above your functions:

```
#include <linux/module.h>
#include <linux/init.h>   // Often needed alongside module.h
```

Why did this happen?

The `module_exit()` statement is a macro provided by the Linux kernel framework. If `<linux/module.h>` isn't included, the compiler has no idea what it is. In older versions of C standards, when a compiler encounters an unknown identifier being used like a declaration, it assumes it must be an integer (`int`), which triggers this exact error.

Check your code structure

Make sure your module file looks like this standard skeleton layout:

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

// 1. Declare your variables at the top (C90 compliance!)
// ...

// 2. Define your cleanup function
static void __exit my_module_cleanup(void) {
    printk(KERN_INFO "Module removed.\n");
}

// 3. Register the exit macro at the bottom
module_exit(my_module_cleanup);

MODULE_LICENSE("GPL");
```

Did adding **`#include <linux/module.h>`** get the code to compile smoothly? Let me know if you run into any other syntax errors!

Copied to clipboardFailed to copy to clipboard. Try again later.

: modpost: missing MODULE_LICENSE() no KO file generated

AI Mode reply for : modpost: missing MODULE_LICENSE() no KO file generated

The error **`modpost: missing MODULE_LICENSE()`** means the Linux kernel build system (`modpost`) rejected your driver because it couldn't find a valid open-source license declaration.

The kernel requires this macro to determine if your module is compatible with the kernel's **GPL** interface. Without it, the compiler **stops and refuses to generate the `.ko` (Kernel Object) file**.

The Fix

Add the **`MODULE_LICENSE()`** macro at the **very bottom** of your `.c` source file, completely

outside of any functions.

Open your `.c` file and add this exact line at the end:

Verify Your Code Layout

Ensure your complete code looks like this basic, correct layout:

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>

static int __init my_init(void) {
    printk(KERN_INFO "Module loaded!\n");
    return 0;
}

static void __exit my_exit(void) {
    printk(KERN_INFO "Module unloaded!\n");
}

module_init(my_init);
module_exit(my_exit);

// ——— ADD THIS LINE AT THE VERY BOTTOM ———
MODULE_LICENSE("GPL");
```

💡 Don't forget to clean before building!

Because the previous build failed during the post-processing phase, run your clean command before trying again:

```
make -C /lib/modules/$(uname -r)/build M=$PWD clean
make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

Did adding **`MODULE_LICENSE("GPL");`** successfully generate your **`.ko` file** this time? Let me know if you see it in your folder when you run `ls`!

Copied to clipboardFailed to copy to clipboard. Try again later.

ERROR: could not insert module hello.ko: Invalid module format

AI Mode reply for ERROR: could not insert module hello.ko: Invalid module format

The error **`Invalid module format`** means the Linux kernel rejected your `.ko` file because it was built for a **different kernel version** than the one currently running on your system.

When you run `insmod`, the kernel checks the **"vermagic"** (version magic) string embedded inside your module. If it doesn't match your active kernel exactly, it blocks execution to prevent a system crash.

Copied to clipboardFailed to copy to clipboard. Try again later.

how to check version number

AI Mode reply for how to check version number

To fix the **"Invalid module format"** error, you need to make sure the kernel version your module was compiled against exactly matches the version of the kernel currently running on your system. If they do not match, the kernel will reject the module for safety.

Here is how you can check both version numbers to find the mismatch:

1. Check the Active Kernel Version

Run this command to see the version of the kernel that is **currently running** on your system:

Example output: `6.8.0-45-generic`

2. Check the Module's Version (The `.ko` file)

Run the **`modinfo`** tool on your compiled module file to see what kernel version it was built for. Look at the **`vermagic`** line in the output:

```
modinfo hello.ko | grep vermagic
```

Example output: `vermagic: 6.5.0-18-generic SMP preempt mod_unload`

How to Fix the Mismatch

If the `uname -r` version and the `vermagic` version do not match, your build command is pointing to the wrong header files.

Copied to clipboardFailed to copy to clipboard. Try again later.

AI Mode reply for they are matching

If the kernel versions **match exactly** but you are still getting `Invalid module format`, the error is usually caused by a mismatch in the **compilation environment** rather than the version string itself.

Here are the three most common reasons this happens and how to fix them:

1. `modpost` is using stale build artifacts

If you changed your code or environment variables during your troubleshooting steps, the build system might be mixing old configuration files with your new code.

You must force a **complete deep clean** of the module directory before compiling again:

```
# 1. Clean out all hidden configurations and cache files
make -C /lib/modules/$(uname -r)/build M=$PWD clean

# 2. Recompile completely fresh
make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

2. A mismatch in Compiler Versions (GCC)

The kernel expects the module to be compiled with the **exact same version of GCC** that was used to compile your running Linux kernel. If your system has

multiple GCC versions installed, it will cause this error.

Check if they match by running these two commands:

```
# Check the compiler version used for your running kernel
cat /proc/version

# Check the compiler version currently being used by your terminal
gcc --version
```

- **The Fix:** If `/proc/version` says the kernel was built with `gcc version 12.2.0` but your terminal is using `gcc version 13.1.0`, you need to pass the correct compiler to your make command:

  ```
  make -C /lib/modules/$(uname -r)/build M=$PWD CC=gcc-12 modules
  ```

3. Missing `Module.symvers` configuration

Sometimes, the kernel configuration settings (`CONFIG_MODVERSIONS`) require explicit module symbol versions. If the symbol table is missing, the kernel rejects the module format.

Try adding a clean target specifically for modules, then rebuilding:

```
make -C /lib/modules/$(uname -r)/build M=$PWD modules_prepare
make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

🔍 How to get the exact error details

If you run the clean and rebuild steps above and `insmod` still fails, the kernel will print the *exact* reason it rejected the format to the system log. Run this command immediately after the failed `insmod hello.ko`:

Look for lines mentioning `hello`. It will usually tell you exactly what is wrong, such as:

- *`hello: disagrees about version of symbol module_layout`* (Symbol mismatch) [1]
- *`hello: unknown symbol...`* (Missing function dependency)

What does the **`dmesg | tail -n 20`** output say right after you try to insert the module? That output will tell us the exact structural reason the kernel is rejecting it.

Copied to clipboardFailed to copy to clipboard. Try again later.

[10179.637634] first_interrupt: loading out-of-tree module taints kernel.
[10179.637737] first_interrupt: module verification failed: signature and/or required key missing - tainting kernel

AI Mode reply for [10179.637634] first_interrupt: loading out-of-tree module taints kernel. [10179.637737] first_interrupt: module verification failed: signature and/or required key missing - tainting kernel

The messages you see in `dmesg` are actually **warnings, not fatal errors**, and they mean your module successfully passed the version check!

When you see `taints kernel` and `module verification failed`, it just means your kernel is running with **Secure Boot** enabled, and your custom module does not have a digital signature trusted by the UEFI firmware. The kernel still attempts to load the module after printing these warnings.

Since you are seeing these logs but `insmod` still failed with `Invalid module format`, it means the kernel aborted the load **immediately after** these warnings due to a strict format structural mismatch.

Let's fix this by cleaning up the environment and bypassing Secure Boot restrictions:

1. The Ultimate "Deep Clean" and Rebuild

Stale build files from your previous compilation attempts (when the code had missing licenses or header bugs) are likely still cached.

Run these commands in order to completely wipe the cache and rebuild using the active kernel's exact environment:

```
# 1. Wipe out every single build artifact
make -C /lib/modules/$(uname -r)/build M=$PWD clean

# 2. Force the kernel to prepare the exact module specs for this directory
make -C /lib/modules/$(uname -r)/build M=$PWD modules_prepare

# 3. Rebuild the kernel object fresh
make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

2. Check for a 32-bit vs 64-bit Architecture Mismatch

Earlier in your logs, your system threw an error looking for `syscall_32.tbl`. If your OS is 64-bit but your `Makefile` or compiler flags are accidentally forcing a 32-bit build target, the kernel will reject the module format.

Run this command to check the architecture of your generated file:

- Look at the output. If it says something like `ELF 32-bit`, but your running system (`uname -m`) says `x86_64`, the format is invalid.
- **The Fix:** Ensure your terminal environment or `Makefile` isn't setting cross-compilation variables like `ARCH=i386`.

3. Verify the Modinfo Output

Run `modinfo` to see what metadata the kernel is actually reading from your compiled binary:

Ensure the `vermagic` string matching your `uname -r` does not contain extra flags (like `SMP preempt mod_unload`) that your running kernel lacks.

Try the **three-step deep clean** above and run `sudo insmod hello.ko` again. If it still fails, what is the **very last line** added to **`dmesg | tail -n 5`** after the failure?

Copied to clipboardFailed to copy to clipboard. Try again later.

should I add below in Make file in "all" make -C /lib/modules/$(uname -r)/build M=$PWD modules_prepare

AI Mode reply for should I add below in Make file in "all" make -C /lib/modules/$(uname -r)/build M=$PWD modules_prepare

No, you should not add `modules_prepare` inside your Makefile's `all:` target.

Running `modules_prepare` modifies the **global kernel headers directory** (`/lib/modules/.../build`), which is owned by `root`. If you put it inside your local Makefile's `all:` target, your regular `make` command will fail because it won't have the permissions to write to system directories.

Instead, your `Makefile` should remain simple, and `modules_prepare` should only ever be run manually **once** in your terminal using `sudo` if your header directory is corrupted.

The Correct `Makefile` Structure

Your `Makefile` should look exactly like this:

```
obj-m += first_interrupt.o

all:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

*(Remember: The indentations before `$(MAKE)` must be a literal **Tab** character, not spaces).*

How to use `modules_prepare` correctly

If you suspect your kernel headers are out of sync or missing configuration files, run it directly in your terminal **with `sudo`** before building your module:

```
# 1. Fix the system kernel headers directory (Only need to do this once)
sudo make -C /lib/modules/$(uname -r)/build modules_prepare

# 2. Clean your local directory
make clean

# 3. Build your module safely as a regular user
make
```

Why did `insmod` fail if the version matched?

Since your `dmesg` showed that the kernel actively read the module name (`first_interrupt`), your kernel version is correct. The `Invalid module format` error is likely happening because **`obj-m`** inside your Makefile is still pointing to your old filename (`hello.o`) instead of your new filename (`first_interrupt.o`).

Ensure `obj-m += first_interrupt.o` matches the exact name of your `.c` file!

Did updating **`obj-m`** to match your actual file name and running a fresh **`make`** get the module to finally insert?

Copied to clipboardFailed to copy to clipboard. Try again later.

file hello.ko hello.ko: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV),

BuildID[sha1]=70cdc8eebb8b3b2f17ab4eb529692747361fc9f0, with debug_info, not stripped

AI Mode reply for file hello.ko hello.ko: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), BuildID[sha1]=70cdc8eebb8b3b2f17ab4eb529692747361fc9f0, with debug_info, not stripped

The `file hello.ko` output shows that your module binary is structurally perfectly fine—it is a correct **64-bit x86-64 relocatable ELF binary**.

Since the binary architecture is correct and your `dmesg` output previously confirmed the kernel is actively reading the module (identifying it as `first_interrupt`), the `Invalid module format` error is being triggered by a **strict kernel configuration mismatch** (like symbol versioning or compiler features).

Let's do a fast verification to see exactly what hidden flag the kernel is rejecting.

1. Match the `vermagic` String Exactly

The kernel reads a hidden configuration string inside your module called `vermagic` and compares it to its own configuration. Let's see if they differ.

Run these two commands and look at the outputs:

```
# 1. Check your module's built-in magic string
modinfo hello.ko | grep vermagic

# 2. Check your active kernel's environment string
uname -v
```

- **What to look for:** Do you see words like `SMP`, `preempt`, or `retpoline` in one string but **missing** from the other? If your running kernel expects `6.x.x-generic SMP preempt` but your module was built as just `6.x.x-generic`, the kernel throws `Invalid module format`.

2. Check for the `module_layout` Disagreement

This is the most common reason for your exact symptoms. If your kernel headers package doesn't perfectly match your running kernel build, the layout of standard internal kernel structures will differ.

Run this immediately after a failed `sudo insmod hello.ko`:

```
dmesg | grep -E "disagrees about version|unknown symbol"
```

- If you see an error like **`hello: disagrees about version of symbol module_layout`**, it means your `linux-headers` package is slightly newer or older than the actual kernel running your machine right now.

The Ultimate Fix if Layouts/Versions Disagree

If the steps above show a mismatch, your running kernel and installed header packages are out of sync (often caused by an OS update that requires a reboot to complete).

Run this to sync your system up, reboot, and try one final clean build:

```
# 1. Ensure you have the absolute latest matching headers
sudo apt update && sudo apt install --reinstall linux-headers-$(uname -r)

# 2. REBOOT your computer to ensure you are booted into the correct kernel
sudo reboot

# 3. Go back to your directory, deep clean, and build
make -C /lib/modules/$(uname -r)/build M=$PWD clean
make -C /lib/modules/$(uname -r)/build M=$PWD modules
sudo insmod hello.ko
```

What does **`modinfo hello.ko | grep vermagic`** output compared to your **`uname -r`**? If you see any trailing words or mismatched text between them, let me know!

Copied to clipboardFailed to copy to clipboard. Try again later.

AI Mode reply for It is virtualbox

Knowing that you are running inside a **VirtualBox virtual machine** explains the mystery.

VirtualBox guest machines often mount standard kernel headers in a way that tricks external compiler tools into compiling against a slightly mismatched structure. Because the `dmesg` logs showed a signature verification error right before breaking, **Secure Boot** emulation or out-of-sync **VirtualBox Guest Additions** are forcing the kernel to block your module execution.

You can resolve this using the following steps:

1. Fix the VirtualBox Header Synced Cache

If you recently ran system updates or installed headers inside the VM, the underlying symbolic link pointing to the build headers might be corrupt. Force-refresh them in order:

```
# 1. Reinstall the clean header packages for your running version
sudo apt update && sudo apt install --reinstall linux-headers-$(uname -r) build-essential dkms

# 2. Reset the module build directory cache
sudo make -C /lib/modules/$(uname -r)/build modules_prepare
```

2. Bypass VirtualBox Module Sign Enforcements

Because your `dmesg` states `module verification failed: signature and/or required key missing`, the VirtualBox VM framework is strictly requiring custom drivers to be crypto-signed via UEFI/MOK. You can bypass this restriction by telling `insmod` to drop strict check validation:

Compile your module fresh:

```
make -C /lib/modules/$(uname -r)/build M=$PWD clean
make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

Now try loading the driver with the **force parameter flag**:

(Alternatively, try `sudo insmod first_interrupt.ko` depending on what your final `.ko` file name ended up as).

3. Check the Exact "Reject Reason" If It Fails

If `insmod -f` still returns an invalid format error, VirtualBox's kernel layer is actively pointing out an interface mismatch. Type this exact line to see the terminal failure payload:

```
sudo dmesg -c > /dev/null   # Wipes the log history
sudo insmod hello.ko        # Fires the insert
dmesg                       # Reads ONLY the new error text
```

Did using **`insmod -f`** successfully bypass the signature check and insert the module into your VirtualBox VM? If you get a specific string from that fresh **`dmesg`** reading, paste it right here!

Copied to clipboardFailed to copy to clipboard. Try again later.

[11216.550642] module: x86/modules: Skipping invalid relocation target, existing value is nonzero for type 1, loc 000000000894c9b5, val ffffffffc098b050

AI Mode reply for [11216.550642] module: x86/modules: Skipping invalid relocation target, existing value is nonzero for type 1, loc 000000000894c9b5, val ffffffffc098b050

The error **`Skipping invalid relocation target, existing value is nonzero for type 1`** is a very specific kernel loading error. It means your module was compiled with a feature called **Position Independent Code (`-fPIC` or `-fpie`)**, which is standard for user-space programs but strictly **forbidden** for Linux kernel modules.

Kernel modules must be compiled as relocatable object files (`-fno-pic`). Because VirtualBox environments or modern Linux distributions often turn on hardened `PIC` settings by default for user space, your compiler is accidentally applying it to your kernel module.

The Fix: Force `-fno-pic` in Your Makefile

You need to explicitly tell the kernel build system to turn off PIC/PIE flags when building your module.

Open your project's **`Makefile`** and add the following two lines right near the top (above the `all:` target):

```
obj-m += hello.o

# ─── ADD THESE TWO LINES ───
ccflags-y += -fno-pic -fno-pie
ldflags-y += -no-pie
# ──────────────────────────

all:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	$(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

(Note: If your `.c` file is named `first_interrupt.c`, make sure `obj-m +=` says `first_interrupt.o` instead of `hello.o`).

Recompile and Load

Once you save the Makefile, run a complete deep clean to wipe out the broken binary cache, rebuild it without the PIC flags, and insert it:

```
# 1. Clean the old compilation artifacts completely
make clean

# 2. Build the module fresh with the new flags applied
make

# 3. Load the module
sudo insmod hello.ko
```

Did forcing **`-fno-pic`** in your Makefile stop the relocation error and let the module load successfully? Wrap up by running **`lsmod | grep hello`** to verify it is running!

Copied to clipboardFailed to copy to clipboard. Try again later.

AI Mode reply for Same error

Since the `-fno-pic` flag in the Makefile didn't stop the error, it means the compiler configuration is being overridden by an underlying toolchain discrepancy.

In Linux kernel environments (especially inside VirtualBox VMs), this exact **`Skipping invalid relocation target... type 1`** error almost always traces back to one of two hidden causes: your environment is accidentally using **`clang`** instead of `gcc`, or your system has a mismatch between the **kernel tools binary** and the `linux-headers` package.

Here is how to definitively bypass this mechanism:

1. Explicitly Force the GCC Toolchain

If your system's compiler defaults are slightly misconfigured, `make` might be passing flags incorrectly. Force it to use clean standard `gcc` settings directly from the terminal prompt:

```
# 1. Clean the environment entirely
make -C /lib/modules/$(uname -r)/build M=$PWD clean

# 2. Rebuild by explicitly passing GCC as the absolute compiler compiler
make -C /lib/modules/$(uname -r)/build M=$PWD CC=gcc modules
```

2. Inject the Flag via the Kernel's Build System

Instead of declaring `ccflags-y` inside the local Makefile text (where the parent kernel framework can sometimes discard it), pass the flag straight through the command invocation chain:

```
make -C /lib/modules/$(uname -r)/build M=$PWD EXTRA_CFLAGS="-fno-pic" modules
```

3. Check for the "Partial Upgrade" Loop (Most Common in VMs)

In VirtualBox setups, a common culprit behind this relocation crash is an interrupted package update where the running kernel is slightly older than the headers package.

Run this command sequence to ensure they are forcefully aligned:

```
# Force sync the headers to your exact boot version
sudo apt-get install --reinstall linux-headers-$(uname -r) build-essential

# Reboot the virtual machine to clear out any stale locked memory regions
sudo reboot
```

Once the VM turns back on, go back into your folder and run `make` fresh.

Did forcing the **`CC=gcc`** or **`EXTRA_CFLAGS="-fno-pic"`** runtime arguments finally clear the relocation type 1 fault? If you execute **`gcc --version`**, what version does your system display?

Copied to clipboardFailed to copy to clipboard. Try again later.

- Kernel modules and -fno-pic - Doozan Forum

  My Makefile was very simple: I had to change it to: EXTRA_CFLAGS=-fno-pic M=$(PWD) modules clean: make

- Nvidia invalid relocation target when performing 'modprobe ...

  Module fails to load. There is definitely a bug here, because observations don't match expectations. * Offering clang as alternati...

- Insmod fails - module does not match running kernel but ...

  5 Feb 2024 — Skipping invalid relocation target, The only way I'm aware of this can occur is that you have different kernel symbol versions in ...

